

UNIVERZITET U NOVOM SADU
FAKULTET TEHNIČKIH NAUKA
DEPARTMAN ZA RAČUNARSTVO I AUTOMATIKU

PROJEKTNA DOKUMENTACIJA

MAVN PREVODILAC

Prevođenje višeg MIPS asemblerskog jezika u MIPS 32-bitni assembler

Kandidat: Anđelija Tošić

Broj indeksa: RA8/2024

Predmet: Osnovi paralelnog programiranja i softverski alati

Mentor: Teodora Novković

Novi Sad, jun 2026.

Sadržaj

Sadržaj.....	Error! Bookmark not defined.
1. Uvod i cilj projekta	3
2. Analiza problema.....	3
3. Koncept rešenja i tok prevođenja	4
4. Leksička analiza i tokenizacija	4
5. Sintaksna i semantička analiza.....	6
6. Međureprezentacija programa	9
7. Graf toka upravljanja.....	9
8. Analiza životnog veka promenljivih.....	11
9. Graf smetnji.....	13
10. Alokacija registara	15
11. Prelivanje registarskih promenljivih	16
12. Generisanje MIPS koda	18
13. Glavni tok programa i verifikacija.....	19
14. Zaključak.....	20

1. Uvod i cilj projekta

Cilj projekta je realizacija prevodioca koji ulazni program napisan u MAVN asemblerskom jeziku prevodi u MIPS32 asemblerski kod. MAVN program koristi virtuelne registarske promenljive, memorijske promenljive, labele i skup podržanih instrukcija, dok se u izlaznom MIPS kodu koriste stvarni fizički registri procesora.

Osnovni zadatak prevodioca je da proveriti ispravnost ulaznog programa, formira unutrašnju reprezentaciju instrukcija, sprovede analize potrebne za dodelu registara i generiše izlaznu .s datoteku. Projekat je proširen dodavanjem tri nove instrukcije: and, or i beq. Instrukcija beq je posebno važna jer uvodi uslovno grananje sa dva moguća naslednika u grafu toka upravljanja.

Prevodiocu je na raspolaganju ograničen broj fizičkih registara, konkretno četiri registra: \$t0, \$t1, \$t2 i \$t3. Zbog toga je neophodno sprovesti analizu životnog veka promenljivih, formirati graf smetnji i izvršiti alokaciju registara. Kada četiri registra nisu dovoljna, koristi se mehanizam prelivanja, kojim se vrednost jedne virtuelne registarske promenljive privremeno čuva u memoriji.

2. Analiza problema

Ulaz u sistem predstavlja tekstualna .mavn datoteka. Ona sadrži deklaracije memorijskih i registarskih promenljivih, deklaraciju funkcije, instrukcije i labele. Izlaz treba da bude validna .s datoteka, pogodna za pokretanje u MIPS simulatoru.

Problem prevođenja nije samo direktna zamena jedne instrukcije drugom. Pre generisanja koda potrebno je proveriti više nivoa ispravnosti: da li su karakteri prepoznatljivi, da li tokeni prate gramatiku jezika, da li su promenljive deklarisanе i pravilnog tipa, kao i da li su labele koje se koriste u grananjima zaista definisane.

Nakon toga se javlja problem dodele registara. Virtuelnih registarskih promenljivih u MAVN programu može biti više od četiri, ali izlazni MIPS kod sme koristiti samo četiri fizička registra. Zbog toga prevodilac mora da utvrdi koje virtuelne promenljive su istovremeno žive i koje ne smeju deliti isti registar.

Ulazni element	Obrada u prevodiocu	Rezultat
Karakter iz .mavn fajla	Leksička analiza	Lista tokena
Lista tokena	Sintaksna i semantička analiza	Promenljive i instrukcije
Instrukcije	CFG i analiza životnog veka	SUCC, PRED, IN, OUT skupovi
IN/OUT skupovi	Graf smetnji i alokacija registara	Dodela \$t0-\$t3
IR sa dodeljenim registrima	Generator koda	MIPS .s fajl

3. Koncept rešenja i tok prevođenja

Resenje je organizovano kao niz faza. Svaka faza koristi podatke prethodne faze i priprema informacije za naredni korak. Ovakva organizacija omogućava lakše testiranje, izolovanje grešaka i jasnije proširenje projekta novim instrukcijama.



Slika 1. Pojednostavljen tok prevođenja MAVN programa.

Najpre se poziva leksički analizator, koji od ulaznog niza karaktera formira tokene. Sintakсни analizator zatim proverava redosled tokena prema gramatici MAVN jezika i istovremeno pravi unutrašnju reprezentaciju programa. Nakon toga se formira graf toka upravljanja, izračunava životni vek promenljivih i na osnovu toga formira graf smetnji.

Graf smetnji koristi se za alokaciju fizičkih registara. Ako je alokacija moguća, generator koda odmah pravi MIPS izlaz. Ako nije moguća zbog nedostatka registara, pokreće se prelivanje. Nakon prelivanja menja se lista instrukcija, pa se analize moraju ponoviti nad izmenjenim programom.

4. Leksička analiza i tokenizacija

Leksička analiza predstavlja prvu fazu prevođenja. Njen zadatak je da ulazni niz karaktera podeli na smislene celine koje se nazivaju tokeni. Token se sastoji od tipa i vrednosti. Tip govori kojoj kategoriji token pripada, dok vrednost predstavlja konkretan tekst pročitan iz ulazne datoteke.

Primer iz programa	Tip tokena	Vrednost tokena
_reg	T_REG	_reg
r1	T_R_ID	r1
;	T_SEMI_COL	;
m2	T_M_ID	m2
add	T_ADD	add

Prepoznavanje tokena realizovano je pomoću konačnog determinističkog automata. Automat čita karaktere jedan po jedan i prelazi iz stanja u stanje. Kada se dođe do finalnog stanja, poznato je da je prepoznat jedan token. Primenjuje se princip najdužeg tokena, pa se čitanje nastavlja dokle god novi karakteri mogu pripadati istom tokenu.

U kodu se tokenizacija pokreće metodom Do(). Ona u petlji poziva getNextTokenLex(), a zatim odlučuje šta se radi sa dobijenim tokenom. Greška se odmah prijavljuje, kraj fajla zaustavlja analizu, dok se razmaci i komentari preskaču.

```

bool LexicalAnalysis::Do()
{
    while (true)
    {
        Token token = getNextTokenLex();

        switch (token.getType())
        {
            case T_ERROR:
                errorToken = token;
                tokenList.push_back(token);
                return false;

            case T_END_OF_FILE:
                tokenList.push_back(token);
                return true;

            case T_WHITE_SPACE:
            case T_COMMENT:
                continue;

            default:
                tokenList.push_back(token);
                break;
        }
    }
}

```

Sama metoda `getNextTokenLex()` čuva poslednje finalno stanje i poziciju poslednjeg karaktera validnog tokena. Kada naredni karakter više ne može da pripada istom tokenu, od karaktera u baferu pravi se objekat klase `Token`. Ako se ne može pronaći nijedno validno finalno stanje, pravi se token tipa `T_ERROR`.

```

if (lastFiniteState != IDLE_STATE)
{
    token.makeToken(programBufferPosition, lastLetterPos, programBuffer, lastFiniteState);
    programBufferPosition = lastLetterPos;
    return token;
}
else
{
    token.makeErrorToken(programBufferPosition + counter - 1, programBuffer);
    return token;
}

```

Rezultat leksičke analize je lista tokena koja se prosleđuje sintaksnom analizatoru. Od tog trenutka naredne faze više ne rade direktno sa karakterima, već sa tokenima kao što su `T_REG`, `T_R_ID`, `T_ADD`, `T_NUM` i slično.

5. Sintaksna i semantička analiza

Sintaksna analiza proverava da li je redosled tokena saglasan pravilima MAVN gramatike. Gramatika koristi terminalne i neterminalne simbole. Terminalni simboli su konkretni tokeni jezika, dok neterminali predstavljaju složenije programske celine.

Neterminal	Značenje
Q	Jedna kompletna naredba i nastavak programa
S	Deklaracija, labelirana instrukcija ili obična instrukcija
L	Ostatak programa ili kraj fajla
E	Jedna MAVN instrukcija

Gramatika koja se koristi u projektu može se zapisati na sledeći način:

```
Q -> S ; L
L -> Q | eof

S -> _mem mid num
S -> _reg rid
S -> _func id
S -> id : E
S -> E

E -> add rid, rid, rid
E -> addi rid, rid, num
E -> sub rid, rid, rid
E -> la rid, mid
E -> li rid, num
E -> lw rid, num(rid)
E -> sw rid, num(rid)
E -> b id
E -> bltz rid, id
E -> nop
E -> and rid, rid, rid
E -> or rid, rid, rid
E -> beq rid, rid, id
```

Analizator je realizovan rekursivnim spustom. Svaki neterminal ima odgovarajucu funkciju u kodu: Q(), S(), L() i E(). Funkcija Q() uvek obrađuje jednu celinu, očekuje tačku-zarez i zatim poziva L() za nastavak programa.

```
void SyntaxAnalysis::Q()
{
    S();
    eat(T_SEMI_COL);
    L();
}

void SyntaxAnalysis::L()
{
    if (m_currentToken.getType() == T_END_OF_FILE)
        eat(T_END_OF_FILE);
    else
        Q();
}
```

Funkcija S() na osnovu trenutnog tokena odlučuje da li se obrađuje deklaracija memorijske promenljive, registarske promenljive, funkcije, labela ili instrukcija. Ako trenutni token nije dozvoljen početak nijednog od ovih oblika, prijavljuje se sintaksna greška.

```
void SyntaxAnalysis::S()
{
    switch (m_currentToken.getType())
    {
        case T_MEM:  parseMemoryDeclaration(); break;
        case T_REG:  parseRegisterDeclaration(); break;
        case T_FUNC: parseFunctionDeclaration(); break;
        case T_ID:   parseLabelledInstruction(); break;
        default:
            if (isInstructionStart(m_currentToken.getType()))
                E();
            else
                reportSyntaxError("Token ne moze biti pocetak naredbe.");
    }
}
```

Funkcija E() prepoznaje konkretnu instrukciju. Instrukcije add, sub, and i or imaju isti sintaksni oblik sa tri registarska operanda, pa se za njih koristi zajednicka pomoćna funkcija. Instrukcija beq je posebna jer ima dva registarska operanda i ciljnu labelu.

```
case T_ADD:
    parseThreeRegisterInstruction(T_ADD, I_ADD);
    break;
case T_AND:
    parseThreeRegisterInstruction(T_AND, I_AND);
    break;
case T_BEQ:
    parseBranchEqual();
    break;
```

Semantičke provjere su u ovom projektu ugrađene u sintakсни analizator. One proveravaju da li promenljiva postoji, da li je deklarirana odgovarajućim tipom, da li je funkcija deklarirana samo jednom, da li su labele jedinstvene i da li svako grananje vodi ka postojećoj labeli. Na primer, instrukcija `la r1, m1` zahteva da `r1` bude registarska promenljiva, a `m1` memorijska promenljiva.

```
Variable* destination = requireVariable(destinationName, Variable::REG_VAR);  
Variable* memoryVariable = requireVariable(memoryName, Variable::MEM_VAR);
```

Kada se obrađuje labelirana instrukcija, labela se najpre čuva u pomoćnoj promenljivoj *m_pendingLabel*. Prilikom kreiranja naredne instrukcije labela se dodeljuje toj instrukciji pomoću *setLabel()*. Na taj način se kasnije u grafu toka upravljanja može pronaći cilj skoka.

```
void SyntaxAnalysis::parseLabelledInstruction()  
{  
    string labelName = m_currentToken.getValue();  
    eat(T_ID);  
    eat(T_COL);  
  
    m_labels.push_back(labelName);  
    m_pendingLabel = labelName;  
    E();  
}  
  
if (!m_pendingLabel.empty())  
{  
    instruction->setLabel(m_pendingLabel);  
    m_pendingLabel.clear();  
}
```

6. Međureprezentacija programa

Nakon uspešne sintaksne i semantičke obrade formira se međureprezentacija programa. Ona omogućava da se program dalje obrađuje kao skup objekata, a ne kao tekst. Dve najvažnije klase su Variable i Instruction.

Klasa	Uloga	Najvažniji podaci
Variable	Predstavlja jednu deklarisanu promenljivu.	ime, tip, pozicija, početna vrednost, dodeljeni registar
Instruction	Predstavlja jednu MAVN instrukciju.	tip, pozicija, DST, SRC, USE, DEF, IN, OUT, SUCC, PRED, labela, cilj grananja

Registarske promenljive iz MAVN koda tretiraju se kao virtuelni registri. Tek u fazi alokacije svakoj od njih se dodeljuje jedan od stvarnih MIPS registara. Memorijske promenljive se ne boje u grafu smetnji, već se ispisuju u .data sekciji generisanog koda.

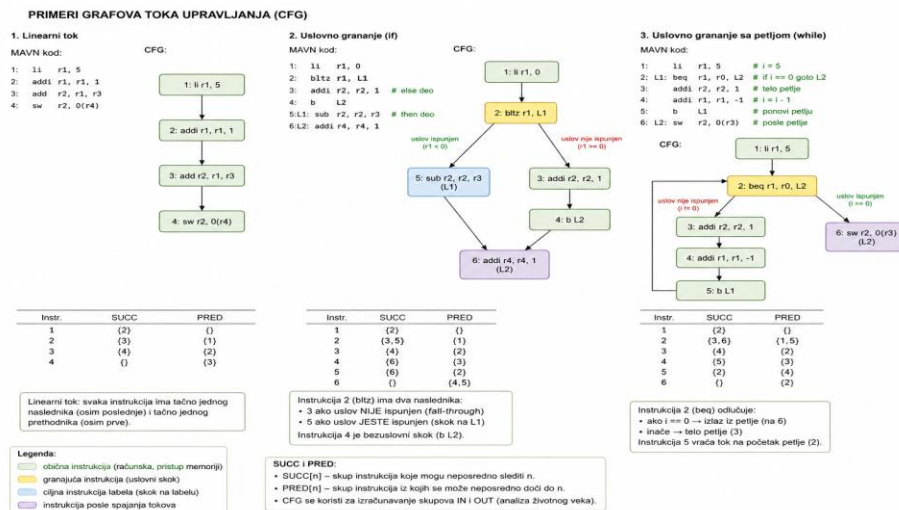
Objekat Instruction čuva i operande instrukcije. Polje DST sadrži odredišne operande, a SRC izvorne operande. Iz njih se formiraju skupovi USE i DEF, koji se kasnije koriste za analizu životnog veka.

Instrukcija	DST	SRC	DEF	USE
add r3,r1,r2	{r3}	{r1,r2}	{r3}	{r1,r2}
li r1,5	{r1}	{}	{r1}	{}
lw r4,0(r3)	{r4}	{r3}	{r4}	{r3}
sw r6,0(r8)	{}	{r6,r8}	{}	{r6,r8}
beq r1,r2,lab	{}	{r1,r2}	{}	{r1,r2}

Razlika između SRC i USE je važna. SRC može sadržati i memorijsku promenljivu, na primer kod instrukcije la r1, m1. Međutim, u USE skup ulaze samo registarske promenljive koje učestvuju u alokaciji fizičkih registara.

7. Graf toka upravljanja

Graf toka upravljanja opisuje kojim redosledom instrukcije mogu da se izvrše. Svaka instrukcija je jedan čvor grafa, a grane pokazuju moguće prelaze. Za svaku instrukciju formiraju se skupovi SUCC i PRED. SUCC sadrži neposredne naslednike instrukcije, dok PRED sadrži instrukcije iz kojih se može doći u trenutnu instrukciju.



Slika 2. Primeri grafa toka upravljanja za linearan tok, grananje i petlju.

Obične instrukcije imaju za naslednika sledeću instrukciju u listi. Bezuslovni skok b ima samo ciljnu instrukciju kao naslednika. Uslovne instrukcije bltz i beq imaju dva naslednika: ciljnu instrukciju i sledeću instrukciju u programu.

```

switch (instruction->getType())
{
case I_B:
    addEdge(instruction, findInstructionByLabel(target));
    break;

case I_BLTZ:
case I_BEQ:
    addEdge(instruction, findInstructionByLabel(target));
    addEdge(instruction, nextInstruction);
    break;

default:
    addEdge(instruction, nextInstruction);
    break;
}

```

Funkcija `addEdge()` konceptualno uvek ažurira oba smera veze: trenutnoj instrukciji dodaje naslednika, a ciljnoj instrukciji dodaje prethodnika. Ovo je važno jer se kasnije OUT skup računa preko SUCC veza, dok PRED veze olakšavaju proveru i ispis grafa.

```

void addEdge(Instruction* from, Instruction* to)
{
    if (to == nullptr)
        return;

    from->getSucc().push_back(to);
    to->getPred().push_back(from);
}

```

Kod petlji se javlja povratna grana. Na primer, ako instrukcija `bltz r7, lab` vodi nazad na instrukciju sa labelom `lab`, tada kasnija instrukcija ima raniju instrukciju kao naslednika. Zbog takvih povratnih veza analiza životnog veka mora biti iterativna.

8. Analiza životnog veka promenljivih

Analiza životnog veka određuje u kojim delovima programa je trenutna vrednost neke registarske promenljive još potrebna. Promenljiva je živa u nekoj tački ako postoji mogućnost da će njena vrednost biti upotrebljena kasnije, pre nego što joj se dodeli nova vrednost.

Za svaku instrukciju koriste se skupovi USE, DEF, IN i OUT. USE i DEF su poznati iz same instrukcije. IN i OUT se računaju na osnovu USE, DEF i SUCC veza iz grafa toka upravljanja.

```
OUT[n] = union IN[s], za svako s iz SUCC[n]
IN[n]  = USE[n] union (OUT[n] - DEF[n])
```

Analiza se vrši unazad jer život promenljive zavisi od njene buduće upotrebe. U jednom prolazu od poslednje ka prvoj instrukciji računaju se nove vrednosti skupova. Ako se neki IN ili OUT skup promeni, analiza se ponavlja. Postupak se zaustavlja kada se dostigne stabilna tačka, odnosno kada se u celom prolazu više nijedan skup ne promeni.

```
void LivenessAnalysis::calculateLiveness()
{
    bool changed;
    do
    {
        changed = false;

        for (auto it = m_instructions.rbegin(); it != m_instructions.rend(); ++it)
        {
            Instruction* instruction = *it;
            Variables oldIn  = instruction->getIn();
            Variables oldOut = instruction->getOut();

            calculateOut(instruction);
            calculateIn(instruction);

            if (!sameVariableSets(oldIn, instruction->getIn()) ||
                !sameVariableSets(oldOut, instruction->getOut()))
            {
                changed = true;
            }
        }
    }
    while (changed);
}
```

Prilikom računanja OUT skupa prolazi se kroz sve naslednike instrukcije i u OUT trenutne instrukcije se dodaju sve promenljive iz IN skupova naslednika. Kod uslovnih grananja to znači da OUT mora obuhvatiti promenljive žive na obe moguće putanje izvršavanja.

```
instruction->getOut().clear();

for (Instruction* successor : instruction->getSucc())
{
    for (Variable* variable : successor->getIn())
    {
        if (!variableExists(variable, instruction->getOut()))
            instruction->getOut().push_back(variable);
    }
}
```

Prilikom računanja IN skupa najpre se dodaju sve promenljive iz USE skupa, jer su one potrebne pre izvršavanja instrukcije. Zatim se dodaju promenljive iz OUT skupa koje nisu u DEF skupu. Promenljive iz DEF skupa se izbacuju zato što trenutna instrukcija zamenjuje njihovu staru vrednost novom.

```
instruction->getIn().clear();

for (Variable* variable : instruction->getUse())
    instruction->getIn().push_back(variable);

for (Variable* variable : instruction->getOut())
{
    if (!variableExists(variable, instruction->getDef()) &&
        !variableExists(variable, instruction->getIn()))
    {
        instruction->getIn().push_back(variable);
    }
}
```

Dobijeni IN i OUT skupovi pokazuju koje promenljive su istovremeno žive. To je direktan ulaz za formiranje grafa smetnji. Ako su dve promenljive istovremeno žive, one ne smeju koristiti isti fizički registar.

9. Graf smetnji

Graf smetnji prikazuje koje virtualne registarske promenljive ne smeju dobiti isti fizički registar. Svaka registarska promenljiva predstavlja jedan čvor. Između dva čvora postoji ivica ako su promenljive u smetnji, odnosno ako njihove vrednosti moraju istovremeno biti sačuvane.

Graf se pravi tek nakon analize životnog veka, jer koristi skupove DEF i OUT. Pravilo je: svaka promenljiva iz DEF skupa povezuje se sa svakom različitom promenljivom iz OUT skupa.

```
void InterferenceGraph::buildInterferenceGraph()
{
    for (Instruction* instruction : m_instructions)
    {
        for (Variable* defined : instruction->getDef())
        {
            for (Variable* live : instruction->getOut())
            {
                if (defined != live)
                    addInterference(defined, live);
            }
        }
    }
}
```

Razlog za ovo pravilo je sledeći: promenljiva iz DEF upravo dobija novu vrednost, a sve promenljive iz OUT moraju ostati sačuvane nakon te instrukcije. Zbog toga definisana promenljiva ne sme biti smeštena u isti fizički registar kao promenljive koje su žive posle instrukcije.

Graf se u projektu čuva pomoću matrice smetnji. Ako je element matrice jednak 1, između dve promenljive postoji smetnja. Ako je jednak 0, smetnja nije pronađena. Matrica je simetrična, jer ako r2 ne sme da deli registar sa r5, onda ni r5 ne sme da deli registar sa r2.

```
void InterferenceGraph::addInterference(
    Variable* first,
    Variable* second)
{
    int i = findVariableIndex(first);
    int j = findVariableIndex(second);

    m_interferenceMatrix[i][j] = __INTERFERENCE__;
    m_interferenceMatrix[j][i] = __INTERFERENCE__;
}
```

Nakon formiranja matrice računa se rang svakog čvora. Rang predstavlja broj drugih čvorova sa kojima je promenljiva povezana. Što je rang veći, to je promenljiva teža za alokaciju, jer ne sme deliti registar sa većim brojem drugih promenljivih.

```
for (int i = 0; i < matrix.size(); i++)
{
    int rank = 0;

    for (int j = 0; j < matrix[i].size(); j++)
        if (matrix[i][j] == __INTERFERENCE__)
            ++rank;

    m_ranks.push_back(rank);
}
```

Rang čvora može biti veći od broja fizičkih registara. Na primer, ako je promenljiva r2 povezana sa r3, r4, r5, r6 i r7, njen rang je 5, iako su dostupna samo 4 fizička registra. Rang meri broj smetnji, a ne broj dostupnih registara.

10. Alokacija registara

Alokacija registara rešava problem dodele četiri fizička MIPS registra virtuelnim registarskim promenljivama. U projektu su dostupni registri \$t0, \$t1, \$t2 i \$t3. Problem se posmatra kao bojenje grafa smetnji, gde svaki fizički registar predstavlja jednu boju.

Dva susedna čvora u grafu smetnji ne smeju dobiti istu boju. Ako dve promenljive nisu u smetnji, mogu deliti isti fizički registar, jer njihove vrednosti nisu potrebne istovremeno.

Algoritam ima dve faze: uprošćavanje grafa i izbor registara. U fazi uprošćavanja trazi se čvor čiji je trenutni rang manji od broja dostupnih registara. Pošto postoje četiri registra, trazi se čvor sa rangom manjim od 4. Taj čvor se označava kao uklonjen iz redukovano grafa i stavlja se na stek.

```
if (!removed[i] && currentDegree[i] < __REG_NUMBER__)
{
    removed[i] = true;
    allocationStack.push(registerVariables[i]);

    for (int j = 0; j < numberOfVariables; j++)
        if (matrix[i][j] == __INTERFERENCE__ && !removed[j])
            currentDegree[j]--;
}
```

Uklanjanje ne znači da se promenljiva briše iz programa. Ona se samo privremeno izbacuje iz redukovano grafa i pamti na steku. Kada se ukloni jedan čvor, smanjuju se trenutni rangovi njegovih suseda, jer se veze sa uklonjenim čvorom više ne računaju u preostalom grafu.

U fazi izbora registara čvorovi se skidaju sa steka obrnutim redosledom. Za svaki čvor proverava se koje registre već koriste njegovi susedi, a zatim se bira prvi slobodan registar.

```
while (!allocationStack.empty())
{
    Variable* variable = allocationStack.top();
    allocationStack.pop();

    bool used[__REG_NUMBER__] = { false };

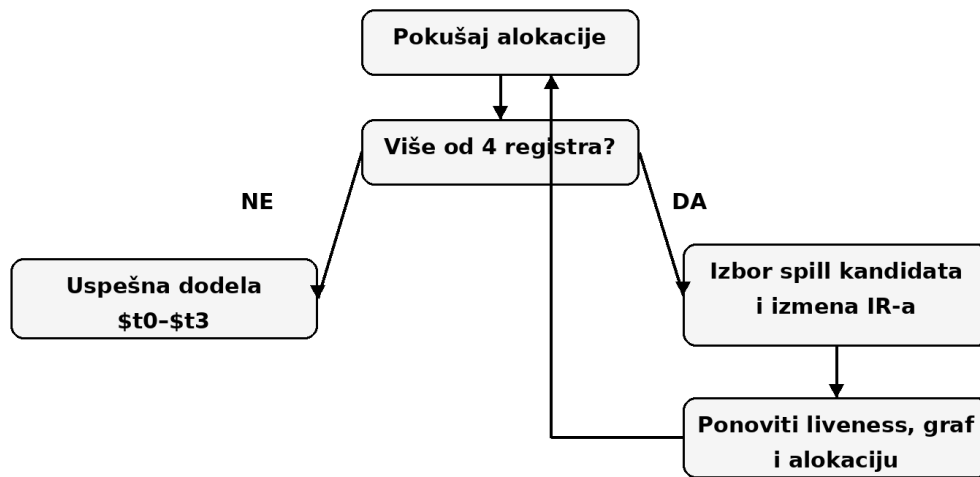
    for (Variable* neighbour : neighbours(variable))
        if (neighbour->getAssignment() != no_assign)
            used[neighbour->getAssignment() - 1] = true;

    variable->setAssignment(firstFreeRegister(used));
}
```

Ako u fazi uprošćavanja ne postoji nijedan čvor sa trenutnim rangom manjim od 4, algoritam ne može da garantuje uspešno bojenje. Tada se detektuje preliivanje i bira se promenljiva koja će se čuvati u memoriji.

11. Prelivanje registarskih promenljivih

Prelivanje se koristi kada graf smetnji nije moguće obojiti sa četiri fizička registra. To se dešava kada je u nekom delu programa potrebno sačuvati više vrednosti nego što postoji fizičkih registara. U programu multiply.mavn javlja se trenutak u kom je istovremeno živo pet registarskih promenljivih, dok su dostupna samo četiri fizička registra.



Slika 3. Tok obrade kada se detektuje prelivanje.

Kada se izabere promenljiva za prelivanje, za njenu vrednost pravi se nova memorijska lokacija, na primer spill_r4_0. Originalna virtuelna registarska promenljiva više ne mora stalno da zauzima fizički registar. Njena vrednost se čuva u memoriji, a u registre se učitava samo kratkotrajno kada je potrebna.

Ako originalna instrukcija definiše prelivenu promenljivu, rezultat se najpre upisuje u privremenu registarsku promenljivu, a zatim se odmah čuva u spill memorijsku lokaciju.

```
// pre prelivanja
lw r4, 0(r3)

// nakon prelivanja
lw temp1, 0(r3)
sw temp1, spill_r4_0
```

Ako se vrednost prelivene promenljive kasnije koristi, ona se prethodno učitava iz memorije u novu kratkoživeću privremenu registarsku promenljivu.

```
// pre prelivanja
sub r7, r5, r4

// nakon prelivanja
lw temp2, spill_r4_0
sub r7, r5, temp2
```

Na ovaj način se jedan dug životni vek promenljive r4 zamenjuje sa više kratkih zivotnih vekova privremenih promenljivih. Vrednost r4 više nije živa kroz veliki deo programa, već se nalazi u memoriji. Privremene promenljive postoje samo oko konkretne upotrebe ili definicije, pa prave mnogo manje smetnji.

U kodu SpillCode prolazi kroz sve instrukcije i proverava da li se prelivana promenljiva javlja u njihovim izvornim ili odredišnim operandima. Ako se koristi, ubacuje se load iz spill memorije pre instrukcije. Ako se definiše, ubacuje se store posle instrukcije.

```
if (usesSpilledVariable)
{
    Variable* temp = createTemporaryRegister("use");
    insertBefore(createSpillLoad(temp, spillMemory));
    replaceVariable(instruction->getSrc(), spilled, temp);
}

if (definesSpilledVariable)
{
    Variable* temp = createTemporaryRegister("def");
    replaceVariable(instruction->getDst(), spilled, temp);
    insertAfter(createSpillStore(temp, spillMemory));
}
```

Nakon transformacije, originalna prelivana promenljiva uklanja se iz liste registarskih promenljivih koje učestvuju u alokaciji. Zatim se ponovo pokreću analiza životnog veka, formiranje grafa smetnji i alokacija registara. Ako ni tada nema dovoljno registara, postupak se može ponoviti.

12. Generisanje MIPS koda

Generator koda se pokreće tek nakon uspešne dodele fizičkih registara. Ulaz generatora predstavljaju lista promenljivih, lista instrukcija, ime funkcije i naziv izlazne datoteke. Prvo se formira .data sekcija, zatim .text sekcija i na kraju se svaka MAVN instrukcija prevodi u odgovarajuću MIPS instrukciju.

```
.data

m1: .word 6
m2: .word 5
spill_r4_0: .word 0

.text
.globl main

main:
```

Kod registarskih operanada generator ne ispisuje više virtuelna imena r1, r2 i slicno, već fizičke registre dobijene alokacijom. Na primer, ako je r3 dodeljen registar \$t0, r1 registar \$t1, a r2 registar \$t2, instrukcija add r3, r1, r2 postaje add \$t0, \$t1, \$t2.

```
case I_ADD:
    output << "\tadd "
        << getRegisterName(dst)
        << ", " << getRegisterName(src1)
        << ", " << getRegisterName(src2)
        << endl;
    break;
case I_BEQ:
    output << "\tbeq "
        << getRegisterName(src1)
        << ", " << getRegisterName(src2)
        << ", " << instruction->getBranchTarget()
        << endl;
    break;
```

Interne spill instrukcije se ne pojavljuju u izvornom MAVN programu. One nastaju samo tokom prelivanja. Generator ih prevodi u MIPS lw i sw instrukcije koje pristupaju spill memorijskoj lokaciji.

```
case I_LW_SPILL:
    output << "\tlw "
        << getRegisterName(destination)
        << ", " << spillMemory->getName()
        << endl;
    break;
case I_SW_SPILL:
    output << "\tsw "
        << getRegisterName(source)
        << ", " << spillMemory->getName()
        << endl;
    break;
```

Pre generisanja je važno proveriti da svaka registarska promenljiva ima dodeljen fizički registar. Ako neka promenljiva nema assignment, generator ne sme nastaviti, jer bi nastao neispravan MIPS kod.

13. Glavni tok programa i verifikacija

Glavni program povezuje sve faze prevodioca. Nakon učitavanja ulaznog fajla pokreće se leksička analiza. Ako ona uspe, pokreće se sintaksna i semantička analiza. Zatim se ulazi u petlju koja pokreće analizu životnog veka, formiranje grafa smetnji i alokaciju registara. Ako se detektuje prelivanje, program se transformiše i petlja se ponavlja.

```
while (!registersAssigned)
{
    LivenessAnalysis liveness(instructions, functionName);
    liveness.Do();

    InterferenceGraph graph(variables, instructions);
    graph.Do();

    ResourceAllocation allocator(graph);

    if (allocator.Do())
    {
        registersAssigned = true;
    }
    else if (allocator.isSpillDetected())
    {
        SpillCode spill(variables, instructions, graph);
        spill.Do();
    }
    else
    {
        throw runtime_error("Dodela registara nije uspela.");
    }
}
```

Ovakav tok je potreban zato što prelivanje menja listu instrukcija. Ubacuju se nove lw i sw instrukcije i nove privremene registarske promenljive. Zbog toga stari IN, OUT i graf smetnji više nisu važeći i moraju se ponovo izračunati.

Verifikacija je izvršena pomoću više ulaznih programa. Program simple.mavn proverava osnovni tok bez prelivanja. Program sa dodatnim instrukcijama proverava and, or i beq. Program multiply.mavn proverava petlju, graf toka upravljanja, analizu životnog veka, graf smetnji, detekciju prelivanja i ponovno pokretanje alokacije.

Test	Sta se proverava	Ocekivani rezultat
simple.mavn	Deklaracije, la, lw, add i osnovna alokacija	Generiše se validan .s fajl bez prelivanja
dodatne_instrukcije.mavn	Instrukcije and, or, beq i grananje ka labeli	Generisu se odgovarajuće MIPS instrukcije
multiply.mavn	Petlja, uslovno grananje, životni vek i spill	Detektuje se prelivanje i nakon transformacije generiše se .s fajl
neispravni ulazi	Leksicke, sintaksne i semantičke greške	Analiza se prekida uz odgovarajuću poruku

Primeri grešaka koje se proveravaju su: nedozvoljen karakter, nedostajući operand, nedostajući znak tačka-zarez, korišćenje nedeklarisane promenljive, pogrešan tip promenljive i grananje ka nepostojećoj labeli.

14. Zaključak

Realizovani prevodilac obuhvata sve ključne faze obrade jednog asemblerskog jezika: leksičku analizu, sintaksnu i semantičku analizu, formiranje međureprezentacije, analizu toka upravljanja, analizu životnog veka, formiranje grafa smetnji, alokaciju registara i generisanje MIPS koda.

Najvažniji deo projekta predstavlja povezivanje analize životnog veka sa alokacijom registara. Na osnovu IN i OUT skupova određuje se koje promenljive su istovremeno žive, pa se formira graf smetnji. Taj graf omogućava bezbednu ponovnu upotrebu fizičkih registara kada se životni vekovi promenljivih ne preklapaju.

U slučaju da četiri fizička registra nisu dovoljna, implementirano je prelivanje. Time se duga živa promenljiva premesta u memoriju, a njene upotrebe i definicije se zamenjuju kratkozivcem privremenim promenljivama. Nakon toga se analize ponavljaju dok graf ne postane moguće obojiti sa dostupnim registrima.

Projekat je proširen novim instrukcijama and, or i beq, čime je pokazano da se postojeće faze mogu nadograditi dodavanjem novih tokena, pravila gramatike, tipova instrukcija i pravila generisanja MIPS koda. Time je dobijeno modularno rešenje koje se može dalje prosirivati novim instrukcijama i dodatnim optimizacijama.